

Simulation

There are 3 types of **riders**, 50% are commuters, 25% are party goers and 25% are sporadic riders. For commuters, half of them have an average request time of 8am, half of them have average request time of 6pm. For party goers, they have an average request time of 10pm. For sporadic riders, they have their request time followed by a normal distribution around the whole day.

In terms of number of requests for each rider, their daily number of requests follows Poisson distribution. Commuters have their lambda of 1(i.e. On average they have 1 request per day), party goers and sporadic riders have their lambda of 0.75(i.e. On average they have 0.75 requests per day).

Their request locations are sampled from a uniform distribution of 0 to 1. In the future, we could have airports and theaters where requests have higher density than elsewhere.

For **drivers**, 80% of them work from 7am to 10pm, 20% of them work the rest of the hours. Their idle start times were drawn from a multinomial distribution with a vector of 24 hours rescaled based on this 80-20 rule.

Their idle times(i.e. Working hours) follow an exponential distribution with $1/8 \cdot 60$ (i.e. On average each driver work for 8 hours)

Trip distances are calculated based on the euclidean distances between the start and end coordinates.

During **matching**, we check every 30 minutes(match_interval_time) for each day. Within each 30 minutes, we match on each one of 16 subblock. If a request and a driver are matched, we calculate the distance and time of this potential ride(we assume a constant drive speed). Then a price is calculated based on an affine function of a constant, the ride distance and the ride time.

The **price** will be exposed to both rider and driver, who will accept the ride based on a calculated probability. The **acceptance probability** for the rider is computed from a sigmoid function of a linear transformation of the distance normalized price(with parameters of a_r , b_r), same idea for driver but with different linear hyperparameters(with parameters of a_d , b_d).

The **reward** is the charging percentage(default is 30%) times the total prices of all the matched trips, which is taken by the ridesharing company. This is the target we are trying to maximize.

Input:

short-term rider's elasticity is around -0.05(i.e. one unit increase of request price will cause 5% decrease of acceptance prob for riders)

a_r = 1.5 # Rider acceptance probability parameter

b_r = -0.2 # Rider acceptance probability parameter

short-term driver's elasticity is around -0.05 as well

a_d = 1.5 # Driver acceptance probability parameter

b_d = -0.3 # Driver acceptance probability parameter

Output:

Every demand, supply, and exposed prices within each week.

Reinforcement Learning(A contextual bandit problem):

<https://docs.google.com/document/d/1Nukrk8BvpvXrCT82z1xGeDFGktpLKaChvmWUxNuXAXc/edit>

To recap, each exposed price is calculated by

$$Price_{ride} = \theta_0 + \theta_1 * minutes + \theta_2 * miles$$

RL but not future state(action now won't affect future state)

Update thetas every day, so 3*7=21 thetas to be updated every week

Actions are the choices of thetas

A detailed overview of this problem:

1. **Policy Function:** The policy is represented by a matrix W of shape (num_actions, context_dim). The policy function is:
thetas = W * context
where context is a vector encoding the day of the week and previous rewards.

Dimension wise: thetas is of (3, 1), W is of (3, 13), and context is of (13, 1)

2. **Context:** The context vector is composed of:
 - One-hot encoding of the day of the week (7 dimensions)
 - Rewards from previous days in the week (6 dimensions)
3. **Actions:** The actions are the three theta values (theta_0, theta_1, theta_2) used to set the price.

4. **Update Rule:** After each day, we update the policy based on the gradient of the reward with respect to each theta:

$W[\text{theta_index}] = W[\text{theta_index}] + \text{learning_rate} * \text{gradient} * \text{context}$
This update is performed separately for each theta_index (0, 1 or 2).

We may put weight decay on the W, which has the following update rule:

$W[\text{theta_index}] = W[\text{theta_index}] + \text{learning_rate} * (\text{gradient} * \text{context} + \lambda_2 * W[\text{theta_index}])$

where λ_2 is the decay factor of the l2 weight decay, default value is 0.01

We put a decay on learning_rate as well:

Learning_rate at week t: $lr_t = lr_0 * (d)^t$

where d is a decay factor with default value 0.95, t starts from 0.

To use this policy in the simulation:

1. At the start of each day:
 - Use the theta value from random initialization or previous day to set prices for rides throughout the day.
2. At the end of each day:
 - Calculate the gradient of the reward with respect to each theta (using the method from the previous implementation).
 - Update the previous_rewards array with the reward from the current day.
3. At the end of the week, reset previous_rewards to zeros.

As an example,

1. Context:

- The context is a 13-dimensional vector.
- The first 7 dimensions represent the day of the week (one-hot encoded).
- The last 6 dimensions represent the rewards from the previous 6 days.

For Wednesday (day 2), with rewards of 1000 on Monday and 1200 on Tuesday, the context looks like:

...

[0, 0, 1, 0, 0, 0, 0, 1000, 1200, 0, 0, 0, 0]

...

2. Policy (W matrix):

- The W matrix has shape (3, 13), where each row corresponds to one of the theta parameters.
- It's initialized with small random values.

3. Getting Thetas:

- Thetas are calculated by multiplying the W matrix with the context vector.
- This allows different weights to be applied to different days of the week and to the reward history.

4. Updating the Policy:

- After observing the performance (gradient) for a day, we update the W matrix.
- The update is performed separately for each theta (action).
- The magnitude of the update depends on the gradient, the learning rate, and the context.

5. Next Day:

- For Thursday, the context shifts: the one-hot encoding moves to the next position, and the reward history is updated.
- This results in different theta values for Thursday, even before any learning has occurred.

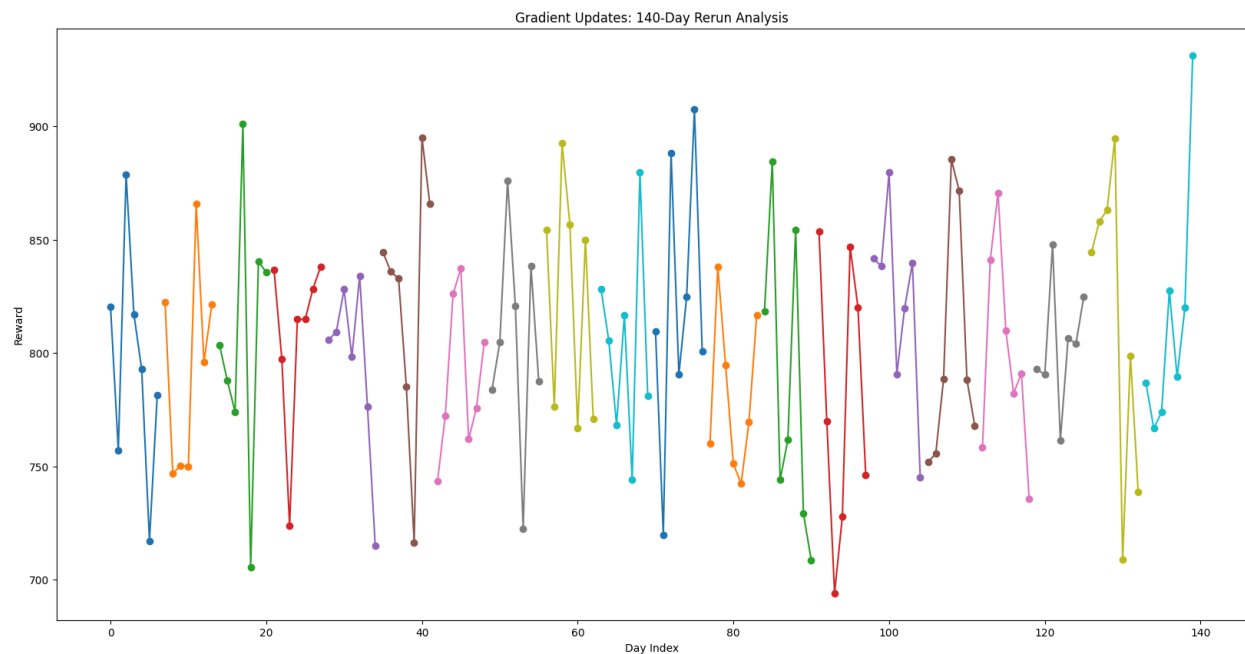
Result and Analysis

20-week simulation data of exposed prices and matched rides:

reinforcement_learning_for_rideshare_pricing/data/update_gradient/20_weeks

Experiment I:

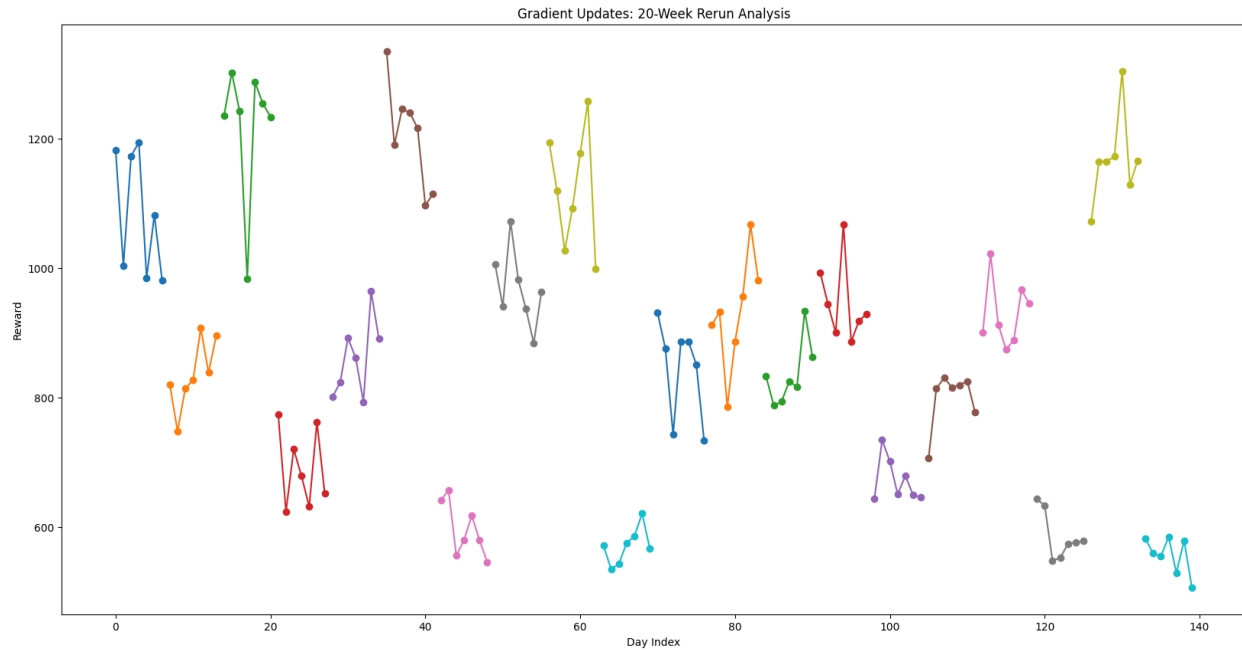
train for 140 days, the simulation carries on(i.e. thetas, l_r will be taken from the previous days)
Context is then (279, 1), W is then (3, 279)



We do need a spike of rewards at the end of the learning, however the mean of the rewards are somewhat flat.

Experiment II:

train for 20 weeks(which has 140 days), each week we reset the simulation(i.e. thetas, lr and such is reset every week).



For some weeks, we do see the rewards get increased after learning, however the learning curve is somewhat random.

Appendix

An example of a demand from a rider(i.e. A rider request):

```
{
  "request_timestamp": 563.2579956054688, #in minutes of a day, from 0 to 24*60-1
  "x_start": 0.8575618863105774, #~U(0, 1) as the x-coordinate of the start position
  "y_start": 0.14165323972702026, #~U(0, 1) as the y-coordinate of the start position
  "x_end": 0.8008968234062195, #~U(0, 1) as the x-coordinate of the end position
  "y_end": 0.2355772852897644, #~U(0, 1) as the y-coordinate of the end position
  "rider_idx": 0.0, #hyper-param: 0 to num_riders-1, default is 1000
  "trip_start_subblock_id": 12.0, #hyper-param: 0 to num_subblock-1, default is 0 to 15
  "trip_end_subblock_id": 12.0, #hyper-param: 0 to num_subblock-1, default is 0 to 15
}
```

An example of a supply from a driver(i.e. A idle driver's position):

```
{
  "idle_start_timestamp": 600.0, #in minutes of a day, from 0 to 24*60-1
  "idle_duration": 478.0, #idle minutes, from 0 to 24*60
  "idle_x_start": 0.3753390908241272, #~U(0, 1) as the x-coordinate of the start position
  "idle_y_start": 0.17536525428295135, #~U(0, 1) as the y-coordinate of the start position
  "driver_idx": 0.0, #hyper-param: 0 to num_drivers-1, default is 100
  "idle_start_subblock_id": 4.0, #hyper-param: 0 to num_subblock-1, default is 0 to 15
}
```

An example of an exposed price(i.e. A price showing to both a rider and a driver based on distance)

```
{
  "price_of_ride": 21.7124, #unit:$
  "distance_normalized_price": 4.4339, #unit:$, price_of_ride / max(1, distance)
  "trip_duration": 10.0, #unit: minute
  "rider_acceptance_prob": 0.6487, #0 to 1
  "driver_acceptance_prob": 0.6487, #0 to 1
}
```

Gradient Calculation

- **ride_gradient:**

$$\text{Ride} = Prob_{ride} * Price_{ride}$$

$$d_{ride}/d_{Price_{ride}} = Prob_{ride} + Price_{ride} * d_{Prob_{ride}}$$

$$\begin{aligned} \text{ride_gradient} &= d_{ride}/d_{Price_{ride}} * d_{Price_{ride}}/d_{\theta} \\ &= 0.3 * (Prob_{ride} + Price_{ride} * d_{Prob_{ride}}/d_{Price_{ride}}) * \text{np.array}([1, 2*\text{miles}, \text{miles}]) \\ &= 0.3 * (Prob_{ride} + Price_{ride} * (d_{Prob_{rider}} * Prob_{driver} + Prob_{rider} * d_{Prob_{driver}})) * \\ &\quad \text{np.array}([1, 2*\text{miles}, \text{miles}]) \\ &= 0.3 * (Prob_{ride} + Price_{ride} * (Prob_{rider} * (1 - Prob_{rider}) * (b_{rider} / \max(1, \text{miles})) * \\ &\quad Prob_{driver} + Prob_{rider} * Prob_{driver} * (1 - Prob_{driver}) * (b_{driver} / \max(1, \text{miles})))) * \\ &\quad \text{np.array}([1, 2*\text{miles}, \text{miles}]) \end{aligned}$$

$$= 0.3 * (Prob_{ride} + Price_{ride} * Prob_{rider} * Prob_{driver} / \max(1, \text{miles}) * ((1 - Prob_{rider}) * b_{rider} + (1 - Prob_{driver}) * b_{driver})) * \text{np.array}([1, 2 * \text{miles}, \text{miles}])$$

0.3 is the percentage that the ride sharing company charges per ride
 $\text{np.array}([1, 2 * \text{miles}, \text{miles}])$ is from the above calculation

$$\text{Since } Prob_{ride} = Prob_{rider} * Prob_{driver}$$

$$\text{Ride_gradient} = 0.3 * Prob_{rider} * Prob_{driver} * (1 + Price_{ride} / \max(1, \text{miles}) * ((1 - Prob_{rider}) * b_{rider} + (1 - Prob_{driver}) * b_{driver})) * \text{np.array}([1, 2 * \text{miles}, \text{miles}])$$

For each ride, we need to acquire $Prob_{rider}$, $Prob_{driver}$, $Price_{ride}$ and miles to compute the gradient.

- **d_prob_ride:**

$$Prob_{ride} = Prob_{rider} * Prob_{driver}$$

$$d_Prob_{ride} / d_Price_{ride} = (d_Prob_{rider} * Prob_{driver} + Prob_{rider} * d_Prob_{driver})$$

- **d_prob_driver:**

$$Prob_{driver} = \sigma(a_{driver} + b_{driver} * Price_{driver} / \max(1, \text{miles}))$$

$$\text{where } a_{driver} = 1.5, b_{driver} = -0.3$$

$$d_Prob_{driver} / d_Price_{driver} = Prob_{driver} * (1 - Prob_{driver}) * (b_{driver} / \max(1, \text{miles}))$$

- **d_prob_rider:**

$$Prob_{rider} = \sigma(a_{rider} + b_{rider} * Price_{rider} / \max(1, \text{miles}))$$

$$\text{where } a_{rider} = 1.5, b_{rider} = -0.2$$

$$d_Prob_{rider} / d_Price_{rider} = Prob_{rider} * (1 - Prob_{rider}) * (b_{rider} / \max(1, \text{miles}))$$

- **Ride:**

$$Price_{ride} = [\theta_0, \theta_1, \theta_2]^T * [1, \text{minutes}, \text{miles}]$$

$$= \theta^T * [1, 2 * \text{miles}, \text{miles}] \text{ where } \theta = [\theta_0, \theta_1, \theta_2]$$

$$d_Price_{ride} / d_ \theta = [1, 2 * \text{miles}, \text{miles}]$$